

LangGraph Integration Report

Komently AI Service

Student: Emircan Gezer

Date: April 19, 2026

Course: AI Systems Integration

1. Project Overview

Komently is an AI-powered comment moderation SaaS. This report documents the integration of **LangGraph** as the orchestration layer, working alongside **CrewAI** multi-agent crews, within the existing FastAPI-based ai-service.

The goal was to add LangGraph *without replacing* CrewAI — both frameworks cooperate: LangGraph handles routing and shared state management, CrewAI handles specialised agent execution.

2. Implementation Steps

- **Defined a shared state schema** — GraphState TypedDict read by both LangGraph nodes and CrewAI bridge code.
- **Built an LLM-powered router node** — GPT-4o-mini classifies each request into moderation, chat, or analyst.
- **Created crew nodes** — each delegates to a CrewAI crew via the `execute_crew_task()` bridge function.
- **Added a finalize node** — standardises all crew outputs before returning to the caller.
- **Compiled the graph** with MemorySaver for in-memory thread-level persistence.
- **Integrated LangSmith** — full tracing of every LLM call and tool use with zero extra instrumentation.

- **Connected to FastAPI** — all three API endpoints invoke the same compiled LangGraph instance.

3. Architecture

Every HTTP request enters through FastAPI and is handed to the LangGraph orchestrator. The router node classifies intent and conditionally dispatches to one of three CrewAI crews. All paths converge at the post-processing node before the response is returned.

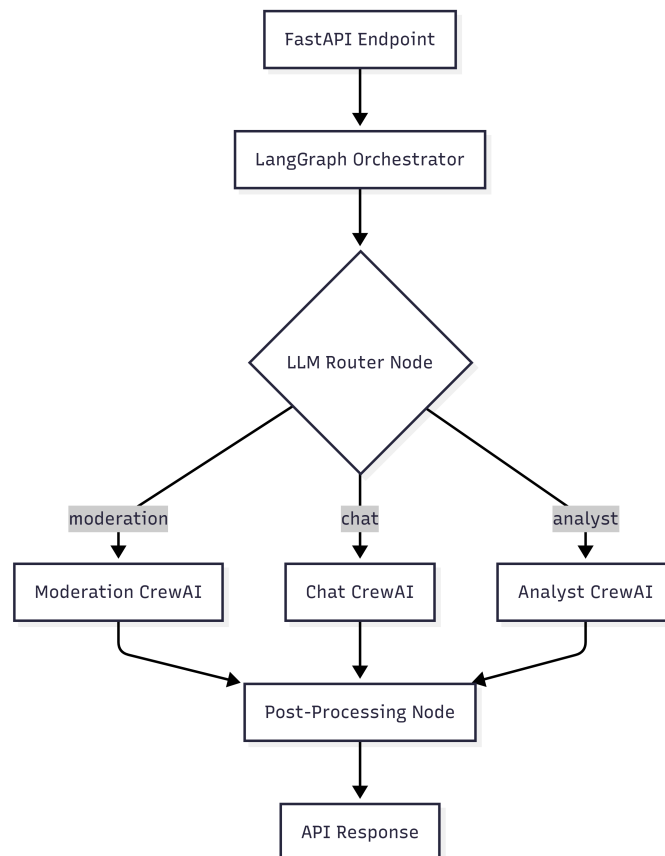


Figure 1: LangGraph + CrewAI Architecture

4. LangGraph Graph (graph.py)

The graph is defined in `graph.py` and exported as the compiled instance `komently_app`. It uses a `StateGraph` with five nodes and a single conditional branch.

GraphState — shared TypedDict

- `input` — raw user message or comment body
- `section_id` — target Komently section
- `history` — conversation turns (used by `/chat`)
- `next_action` — routing decision set by the router node
- `crew_output` — parsed result from the crew
- `final_response` — cleaned string returned to the caller

- metadata — origin endpoint, comment_id, tracing flags

Nodes

Node	Function	Purpose
router	intelligent_router()	GPT-4o-mini classifies the request
moderator	moderation_node()	Runs ModerationCrew
chat	chat_node()	Runs ChatCrew
analyst	analyst_node()	Runs AnalystCrew
finalize	post_process_node()	Standardises all outputs

Edges: A conditional edge from router reads next_action and dispatches to the matching crew node. All three crew nodes converge at finalize, then END.

5. CrewAI Integration (crew.py)

The bridge between frameworks is execute_crew_task() in graph.py. It maps LangGraph state fields into a flat CrewAI input dict, calls .kickoff(), and writes the raw result back into state.

Crew	Agents	Purpose
ModerationCrew	Fetcher → Manager → Moderator (sequential)	Reads settings, builds rulebook, outputs moderation
ChatCrew	Manager (single agent)	Dashboard queries, config changes, etc
AnalystCrew	Analyst (single agent)	Fetches 7-day analytics, writes MarkDown

6. API Endpoints (main.py)

All three endpoints import and invoke komently_app with a unique thread_id for session persistence.

Endpoint	Method	Crew	Response
/moderate	POST	ModerationCrew	JSON: status, toxicity score, isSpam
/chat	POST	ChatCrew	Text reply + actions taken list
/generate-report	POST	AnalystCrew	202 Accepted (background task)

7. LangSmith Integration

Two environment variables in graph.py enable full tracing of every LLM call and CrewAI tool use — no additional instrumentation required.

```
os.environ["LANGCHAIN_TRACING_V2"] = "true" os.environ["LANGCHAIN_PROJECT"] = "Komently-Advanced-Orchestrator"
```

Every invocation — router classification, crew agent reasoning, Supabase tool calls — is recorded in the **Komently-Advanced-Orchestrator** LangSmith project.

8. Key Files

File	Role
graph.py	LangGraph graph: state, nodes, edges, compilation
crew.py	CrewAI crew and agent definitions
main.py	FastAPI app — imports and invokes komently_app
config/agents.yaml	Agent roles, goals, backstories
config/tasks.yaml	Task descriptions and expected outputs
tools/supabase_tools.py	Supabase tools used by Fetcher, Manager agents
tools/analyst_tools.py	Analytics tools used by Analyst agent

9. Git Repository

The full implementation is committed and available at: <https://github.com/emircan-gezer>